

Modelos estocásticos y redes neuronales (versión reducida)

May 29, 2026

1 Modelos estocásticos y redes neuronales

Materia: Análisis de Series de Tiempo y Pronósticos (1C-2026)

Grupo: 9

Integrantes:

- Lucas Achaval - Email: lachavalrodriguez@estudiantes.unsam.edu.ar
- Marcos Achaval - Email: machavalrodriguez@unsam-bue.edu.ar

Título del entregable: Modelos estocásticos y redes neuronales

Conjunto de datos original: dataset propio de una estación meteorológica en un club náutico de Potrerillos (disponibles en [Windguru](#)).

1.1 Resumen

Buscamos pronosticar el viento promedio horario (`wind_avg`) en Potrerillos, Mendoza, a 12 horas, evaluado con MAE, RMSE y R^2 . La primera entrega caracterizó la serie (ciclo diurno marcado, estacionariedad por ADF, picos de ACF/PACF en lags 1-2 y 20-24) y entrenó un LassoCV (24 lags, 6 exógenas, hora cíclica) que redujo el RMSE ~**25 %** y elevó el R^2 de 0.117 a 0.498 frente a la persistencia estacional, sin sobreajuste. Esta entrega suma benchmark, modelos estocásticos y redes neuronales.

1.2 Carga y preprocesamiento

Replicamos el preprocesamiento del primer entregable. Leemos el archivo crudo, remuestreamos por hora (mediana para variables lineales, media circular para `wind_direction`) y agregamos el atributo auxiliar `interval`. Descartamos `wind_min` (completamente vacía) y `gustiness` (derivada de `wind_avg` y `wind_max`).

1.2.1 Split de entrenamiento y test

Separamos el **80 % inicial cronológico** como entrenamiento y el **20 % final** como test. La búsqueda de hiperparámetros (orden de SARIMA, arquitectura de las NN) y los diagnósticos se hacen solo sobre el train; el test se reserva para reportar el desempeño final de todos los modelos bajo la misma metodología.

Completamos los 276 faltantes (4.5 %) con `ffill().bfill()`. Definimos las constantes `HORIZONTE = 12` (pasos a predecir) y `PERIODO = 24` (estacionalidad horaria).

1.3 Modelo de referencia (benchmark)

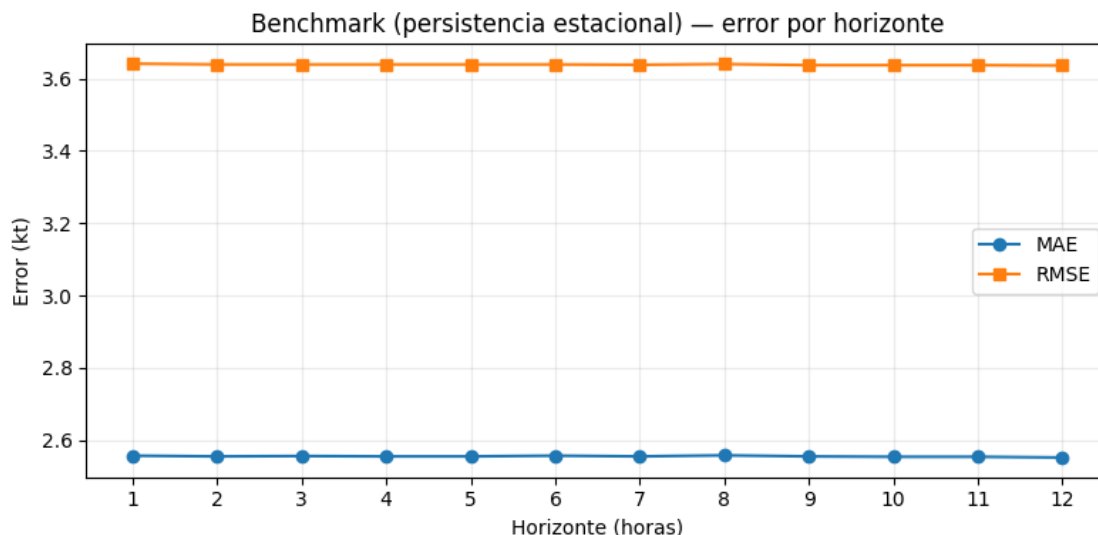
Como benchmark usamos la **persistencia estacional** de período 24 h. Para un instante t y horizonte $h \leq 24$:

$$\hat{y}_{t+h} = y_{t+h-24}$$

Cada hora se pronostica con el valor de la misma hora del día anterior. Lo elegimos porque:

- **Es simple**, sin parámetros que estimar.
- **Aprovecha la única estructura fuerte de la serie**. El ACF del entregable 1 mostró un pico claro en lag 24 (≈ 0.5); una persistencia simple ($\hat{y}_{t+h} = y_t$) lo ignoraría y rendiría peor.
- **Es una línea de base no trivial** que todo modelo más complejo debe superar.

Lo evaluamos sobre `serie_test` con **walk-forward**: para cada origen t pronosticamos a 1, ..., 12 h y comparamos con lo observado, la misma metodología que usaremos en SARIMA y las redes para que las tablas sean comparables.



1.3.1 Desempeño del benchmark

MAE, RMSE y R^2 del benchmark por horizonte (1 a 12 h). Observaciones:

- **Error casi constante con el horizonte**. Como la predicción siempre usa un dato de 24 h atrás, el error apenas cambia dentro de las próximas 24 horas. Este perfil plano contrasta con los modelos de memoria corta (SARIMA, redes), que se degradan al alejarse el horizonte.
- **R^2 positivo pero bajo**. La estacionalidad diaria sola explica poca varianza, dejando margen a los modelos posteriores vía lags cercanos y exógenas.

Este es el desempeño mínimo que deben superar los demás modelos.

1.4 Modelo estocástico

Buscamos un modelo estocástico para la dinámica de `wind_avg` a partir de la ACF y la PACF. El entregable 1 dejó tres hechos relevantes:

- ADF sobre la serie original, estadístico -11.4 y p -valor 7.6×10^{-21} , rechaza la raíz unitaria. **La serie es estacionaria.**
- ACF con pico claro en lag 24; PACF con lags 1-2 y 20-24.
- Espectro dominado por 1 ciclo/día, con un segundo pico más débil en 2 ciclos/día.

1.4.1 ¿Por qué SARIMA y no AR, MA, ARMA o ARIMA?

La elección se desprende de la serie. Según las reglas de las guías 4 y 5:

Modelo	ACF esperado	PACF esperado
$AR(p)$	decae	corta en lag p
$MA(q)$	corta en lag q	decae
$ARMA(p, q)$	decae	decae

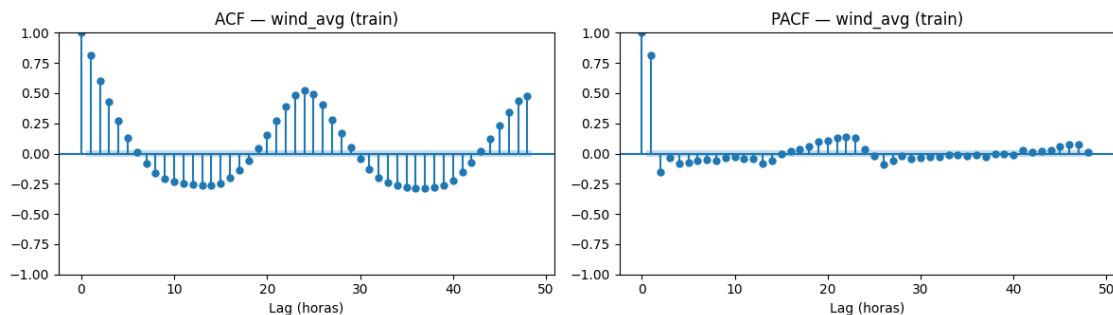
- **AR puro.** La PACF no corta limpio (clusters en 1-2 y 20-24); capturar el pico de lag 24 exigiría $p = 24$, contra la parsimonia.
- **MA puro.** La ACF no corta en lags cortos, decae con oscilaciones (lag 12 día/noche, pico en lag 24).
- **ARMA.** No incorpora período; el pico aislado en lag 24 es estacional y requeriría muchos coeficientes consecutivos.
- **ARIMA.** El ADF confirma estacionariedad, así que $d = 0$ y colapsa al ARMA ya descartado.
- **SARIMA.** Modela la estacionalidad con un único término sobre $\mathbf{B}^{24}$ (guía 5):

$$\Theta_P(\mathbf{B}^{24}) \Theta_p(\mathbf{B}) (1 - \mathbf{B}^{24})^D (1 - \mathbf{B})^d x_t = \Phi_Q(\mathbf{B}^{24}) \Phi_q(\mathbf{B}) w_t$$

Tomamos $d = 0$ (ADF) y $D = 0$ (el pico estacional ya está sin diferenciar); si los residuos muestran estructura cerca de múltiplos de 24, reconsideramos $D = 1$. Exploramos entonces **SARIMA(p,0,q)(P,0,Q,24)**, dejando p, q, P, Q a la búsqueda por AIC sobre `serie_train`.

1.4.2 Funciones de autocorrelación sobre el training

Repetimos el análisis del entregable 1 **solo sobre el training**, para no mirar el test al elegir hiperparámetros. Cualquier inspección que defina el modelo (orden, estacionalidad) debe basarse únicamente en datos vistos.



El training reproduce el entregable 1: PACF significativa en lags 1-2 y 20-24, ACF de decaimiento lento con pico positivo en lag 24. Esto confirma SARIMA y define la grilla:

- $p \in \{0, 1, 2\}$ (PACF hasta lag 2)
- $q \in \{0, 1\}$ (MA corto opcional)
- $P, Q \in \{0, 1\}$ (un término estacional basta para el pico de lag 24)
- $d = D = 0, s = 24, \text{trend} = 'c'$

Son $3 \times 2 \times 2 \times 2 = 24$ combinaciones, acordes a la parsimonia.

1.4.3 Selección del orden con el criterio de Akaike (AIC)

Iteramos la grilla y, por combinación, ajustamos un SARIMA con `statsmodels.tsa.arima.model.ARIMA` y registramos el AIC:

$$\text{AIC} = -2 \log \mathcal{L} + 2k$$

con $\mathcal{L}$ la verosimilitud y k el número de parámetros. Minimizarlo elige el modelo más simple compatible con los datos. Envolvemos el ajuste en `try/except` para descartar combinaciones que no converjan (guía 5).

La tabla ordena las 24 combinaciones por AIC creciente. Tomamos como ganador la primera fila (AIC mínimo).

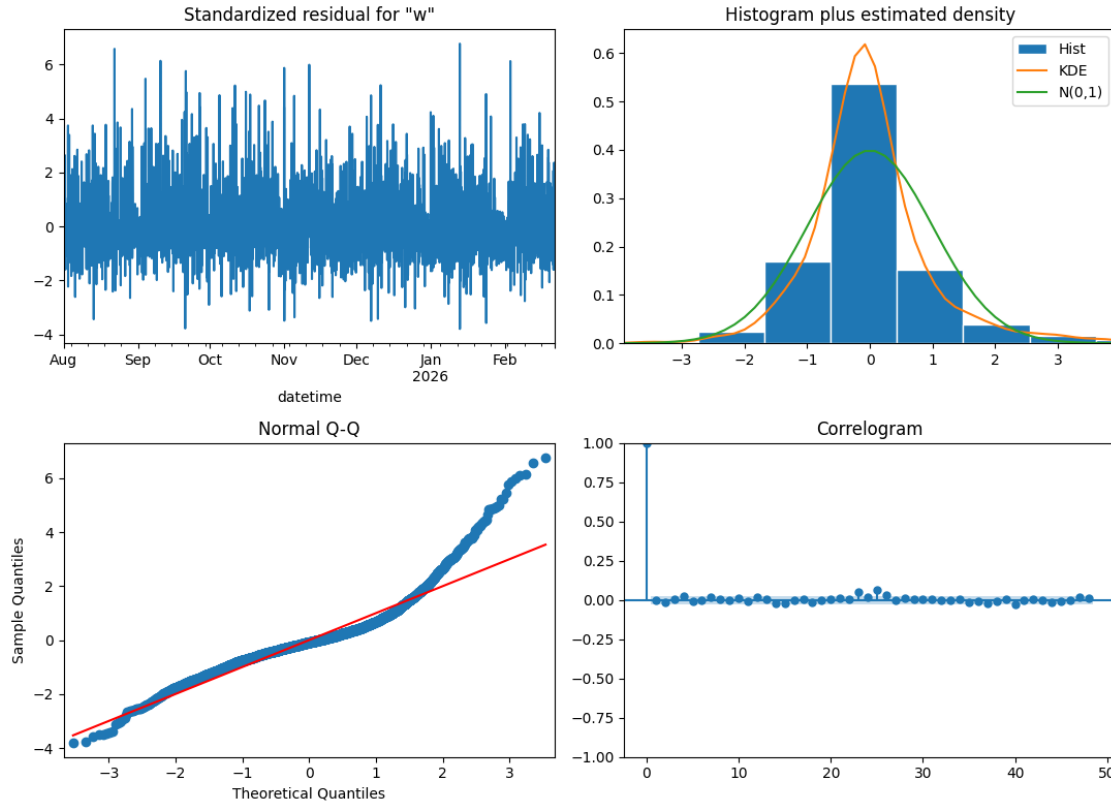
1.4.4 Ajuste del modelo ganador

Refitteamos el orden ganador y revisamos los coeficientes con `summary()`. Buscamos que todos los términos sean significativos ($P > |z|$ cerca de 0) y que las raíces de los polinomios caigan fuera del círculo unitario (estacionariedad e invertibilidad).

1.4.5 Diagnóstico de residuos

Para validar el orden revisamos los residuos. Si el modelo capturó toda la estructura, deben comportarse como ruido blanco (independientes, media cero). Usamos `plot_diagnostics()` (guía 4), que agrupa:

1. **Residuos en el tiempo.** Sin tendencia, heterocedasticidad ni outliers sistemáticos.
2. **Histograma + KDE vs $\mathcal{N}(0, 1)$.** Deberían aproximar una normal.
3. **Q-Q plot.** Puntos sobre la diagonal; desvíos en las colas indican colas pesadas.
4. **Correlograma.** Todos los lags dentro de la banda, sin estructura sobreviviente.



Interpretación del diagnóstico:

- **Residuos en el tiempo.** Oscilan en torno a 0 sin tendencia ni cambio de varianza; hay picos aislados (magnitud 4-7) sin clusters, así que no hay heterocedasticidad fuerte.
- **Histograma + KDE.** La distribución empírica no es gaussiana (más masa cerca de 0 y en las colas, menos en el medio), aunque la varianza total se mantiene cerca de 1.
- **Q-Q plot.** Buen ajuste en el centro y la cola izquierda, pero la cola derecha se dispara (Sample Quantiles de 6-7 frente a Theoretical de 3). Indica asimetría positiva: el modelo capta los vientos bajos y normales pero subestima los picos altos (rachas, eventos térmicos), coherente con la asimetría física del viento.
- **Correlograma.** Casi todos los lags 1-48 caen dentro de la banda del 95 %; los lags 23 y 25 asoman apenas, lo esperable por azar con 48 lags, sin justificar complicar el modelo. El SARIMA absorbió la estructura temporal y el orden elegido por AIC queda validado.

El modelo es válido para pronóstico puntual: se cumple la condición de ruido blanco, hipótesis crítica del ajuste por máxima verosimilitud. La no-normalidad de los residuos es esperable en series meteorológicas y no invalida los coeficientes, pero debe tenerse en cuenta en los intervalos de confianza, que bajo el supuesto gaussiano subestiman los eventos extremos.

Queda definido el modelo estocástico de referencia; lo evaluaremos sobre `serie_test` con el mismo walk-forward del benchmark.

1.5 Evaluación del modelo estocástico

1.5.1 Metodología (windowing)

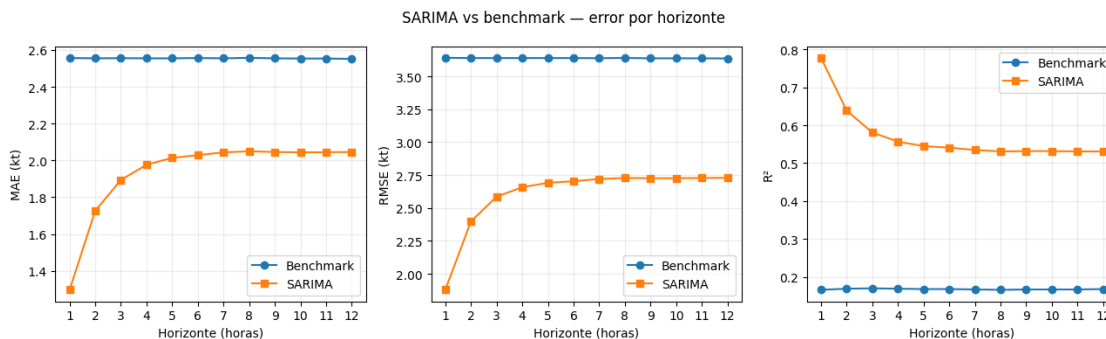
Evaluamos SARIMA con la estrategia del benchmark. Ajustamos una vez sobre `serie_train` y reservamos `serie_test`. La guía 5 usa `extend()` para predicciones a 1 paso; como necesitamos métricas hasta $h = 12$, extendemos esa idea con un **walk-forward** sobre el test, origen por origen:

1. En cada origen t pronosticamos a 12 h con `forecast(steps=12)` y registramos $\hat{y}_{t+1}, \dots, \hat{y}_{t+12}$.
2. Antes del siguiente origen incorporamos la observación con `append(refit=False)`, **sin re-estimar coeficientes**, dejando el filtro de Kalman al día con el último dato (como en operación).

Este patrón `forecast` $\rightarrow$ `append(refit=False)` es el recomendado por statsmodels para walk-forward eficiente y la extensión natural de la guía 5 a multi-paso.

¿Por qué no re-fittear en cada origen? Re-estimar en los ~1200 orígenes implicaría ajustar SARIMA estacional más de mil veces, con un costo enorme para una mejora marginal. Lo habitual es entrenar al inicio y solo actualizar el estado.

Calculamos MAE, RMSE y R^2 por horizonte con la misma `metricas_por_horizonte`, para que las tablas sean comparables.



1.5.2 Interpretación de los resultados

Comparamos SARIMA contra el benchmark en MAE, RMSE y R^2 por horizonte:

- **SARIMA supera al benchmark en todos los horizontes.** Las curvas no se cruzan; aun en el horizonte más largo su RMSE queda por debajo y su R^2 es varias veces mayor.
- **Degradación inicial y meseta desde $h \approx 6$.** El RMSE crece rápido y luego se estanca. El bloque AR(2)/MA(1) decae en ~5-6 pasos, mientras el estacional (1, 0, 1, 24) (con `ar.S.L24` cercano a 1) actúa como ciclo estable; a horizontes largos la predicción converge a la media condicional más el ciclo diario.
- **La meseta queda por encima del benchmark.** Ambos usan el ciclo de 24 h, pero SARIMA con una estimación suavizada y el benchmark con un único dato ruidoso de ayer; esa diferencia de varianza explica la brecha.

La línea de base sube respecto al benchmark, así que las redes deberán superar estas cifras.

1.6 Modelo de redes neuronales

Para pronosticar `wind_avg` a 12 h armamos redes siguiendo la guía 7 (LSTM endógeno). A diferencia de SARIMA, una red no asume linealidad y aprende una función arbitraria de una ventana pasada hacia los 12 pasos futuros, lo que exige dos preprocesos extra:

- **Normalización** de la entrada, para que el optimizador trabaje en escala controlada y los pesos no exploten.
- **Ventaneo**, para convertir la serie en pares (X_i, y_i) que la red consuma como muestras.

Mantenemos el split 80/20, el horizonte de 12 h y la frecuencia horaria. Probamos **tres configuraciones**: una densa de referencia, una LSTM simple con dropout y una LSTM apilada con `return_sequences=True`.

1.6.1 Normalización y ventaneo

Normalización Z-score con estadísticos del train. Restamos media y desvío calculados **solo sobre `serie_train`** y aplicamos la misma transformación al test, evitando filtrar información y dejando la entrada centrada con varianza 1. Casteamos a `float32` (dtype por defecto de Keras).

Ventaneo sequence-to-vector multi-paso. Con paso 1, para cada origen i formamos:

- $X_i = (x_i, \dots, x_{i+W-1})$, los W valores pasados,
- $y_i = (x_{i+W}, \dots, x_{i+W+H-1})$, los $H = 12$ futuros a predecir.

Tomamos $W = 48$ para abarcar **dos ciclos diarios** (guía 7). La última capa densa tiene 12 unidades y predice los 12 horizontes **de una vez** (*direct multi-step*, ejercicio 7), más simple que el esquema recursivo y sin error acumulado, lo que además ordena la comparación con SARIMA.

1.6.2 Arquitecturas propuestas

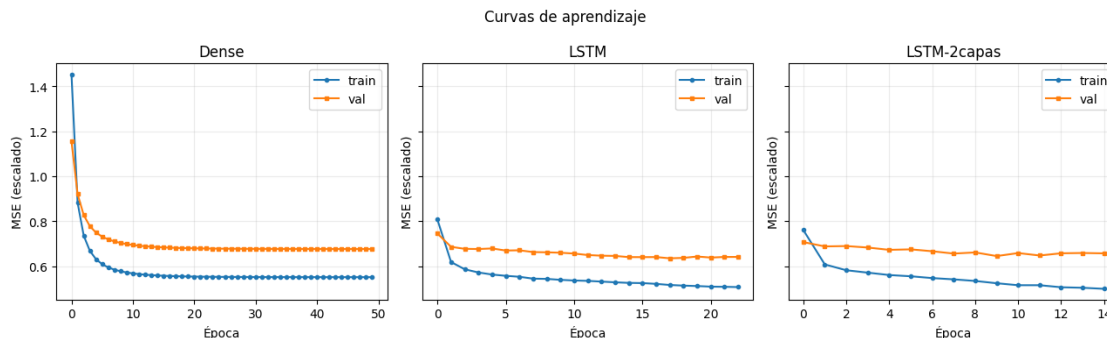
Las tres configuraciones comparten entrada (`Input((48, 1))`) y salida (`Dense(12, activation='linear')`) y difieren en el medio:

- **Dense.** `Flatten` aplana la ventana de 48 pasos y un `Dense(12)` aprende coeficientes lineales hacia cada horizonte. Es una regresión lineal multi-objetivo sobre los lags y sirve de referencia para medir el aporte de la estructura recurrente.
- **LSTM.** Una `LSTM(32)` recorre la ventana y entrega su estado oculto final (32 features), seguida de `Dropout(0.2)` (apaga al azar el 20 % de las unidades para regularizar) y el `Dense(12)`. Es la arquitectura típica de la guía 7.
- **LSTM-2capas.** Apila dos LSTM; la primera con `return_sequences=True` entrega la secuencia completa de estados a la segunda (ejercicio 5), cada una con `Dropout(0.2)`. Mide si más profundidad capta dinámica no lineal adicional.

Optimizador Adam, pérdida MSE (guía 7) y semilla 99 para reproducibilidad.

1.6.3 Entrenamiento

Entrenamos cada modelo sobre `X_train_nn` con `validation_split=0.1` (último 10 % del train, en orden temporal) y `EarlyStopping(patience=5)` sobre la pérdida de validación con restauración de los mejores pesos (ejercicio 4). Dejamos `epochs=50` como tope y la convergencia decide cuándo parar.



1.6.4 Resumen del ajuste

Modelo	Parámetros	Épocas ajustadas	val_loss mínima
Dense	588	50 (tope)	0.676
LSTM	4 748	23	0.635
LSTM-2capas	13 068	15	0.645

Lecturas de las curvas de aprendizaje:

- **Las dos LSTM cortaron por EarlyStopping** (23 y 15 épocas); la Dense llegó al tope de 50 sin cortar, con su pérdida aún bajando, así que su 0.676 es una cota superior.
- **La LSTM simple logra la menor val_loss** (0.635 vs 0.645 apilada y 0.676 densa). La diferencia con la apilada (~ 0.010) viene de una sola semilla y podría ser ruido; una comparación firme requeriría promediar varias semillas. Con esta corrida, las dos LSTM quedan claramente bajo la Dense y entre sí no son concluyentes.
- **Train y validación divergen poco**, señal de que el dropout 0.2 regulariza sin sobreajuste agresivo.

Las pérdidas están en escala normalizada y promediadas sobre los 12 horizontes, no comparables con el RMSE en kt. En la siguiente sección las redes pasan por el mismo walk-forward para reportar métricas en kt.

1.7 Evaluación de la red neuronal

1.7.1 Modelo elegido y metodología

Elegimos la **LSTM simple**: menor val_loss (0.635 vs 0.645 y 0.676) y, frente a la apilada, igual desempeño con menos de la mitad de parámetros (4 748 vs 13 068). Por parsimonia tomamos la más chica.

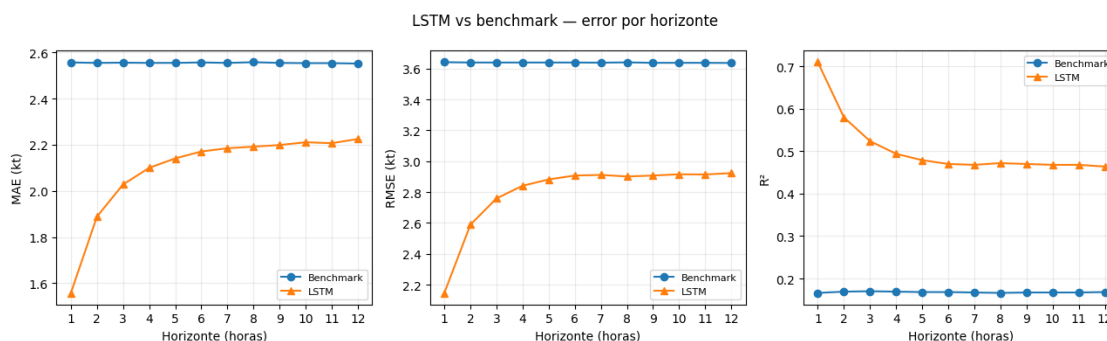
La evaluamos sobre `serie_test` con la **misma metodología que SARIMA**, reportando MAE, RMSE y R^2 por horizonte (1-12 h) en kt contra el benchmark. La comparación con SARIMA y la elección operativa quedan para el punto 8.

El formato es *direct multi-step*: una pasada de `predict()` sobre `X_test_nn` devuelve los 12 horizontes. Cada fila j es un origen cuya predicción usa solo los 48 valores previos, equivalente al loop

forecast → append(refit=False) de SARIMA.

Dos detalles:

- **Desnormalización antes de medir.** El modelo trabaja en escala z ; recuperamos kt con $y * \text{train_std} + \text{train_mean}$ para que las tablas sean comparables.
- **Offset respecto de SARIMA.** El ventaneo arranca en `serie_test.iloc[WINDOW_SIZE + h - 1]` y SARIMA en `serie_test.iloc[h - 1]`, así que la LSTM se evalúa sobre 1170 orígenes y SARIMA sobre 1218; la diferencia es chica y los promedios siguen comparables.



1.7.2 Interpretación de los resultados

Comparamos la LSTM contra el benchmark por horizonte:

- **La LSTM supera al benchmark en todos los horizontes.** En $h = 1$ el salto es grande (RMSE 2.15 vs 3.64 kt, R^2 0.71 vs 0.17): la red aprovecha los lags cercanos de la ventana de 48 h, algo que la persistencia (un dato de 24 h atrás) no puede.
- **Degradación inicial y meseta desde $h \approx 6$.** El RMSE pasa de 2.15 a ~2.9 kt entre $h = 1$ y $h = 6$ y se estanca (R^2 de 0.71 a ~0.47), el mismo perfil que SARIMA.
- **La meseta queda bajo el benchmark.** Aun en $h = 12$ la LSTM (RMSE 2.92, R^2 0.46) gana a la persistencia (3.64, 0.17), apoyándose en una versión suavizada del ciclo diario.

La LSTM supera con claridad la línea de base. En el punto 8 la comparamos con SARIMA.

1.7.3 ¿Los coeficientes son interpretables como importancia de atributos?

No de forma directa. Las redes tienen entre 588 (Dense) y 13 068 pesos, y la relación peso-importancia no es la de una regresión lineal.

- **Dense** es el único caso donde los 588 pesos del `Dense(48 → 12)` mapean $\{\text{lag de entrada}\} \times \{\text{horizonte de salida}\}$, leíbles en principio como matriz de coeficientes. Aun así, la magnitud no equivale a importancia: la entrada está normalizada y los 48 lags están muy correlacionados (autocorrelación alta en 1-2 y 24), lo que reparte el crédito entre lags equivalentes.
- **LSTM y LSTM-2capas** no admiten esa lectura. Sus pesos viven en matrices $(d_{\text{in}} + d_h) \times 4d_h$ que mezclan no linealmente input, estado oculto y los cuatro *gates* a lo largo de 48 pasos. No hay correspondencia uno-a-uno peso-lag; la importancia depende de toda la trayectoria de estados ocultos.

1.8 Redes neuronales con variables exógenas

Hasta acá la red usó **solo el pasado de wind_avg**. Ahora sumamos exógenas (guía 8, *LSTM-exo*): cada paso de la ventana pasa de escalar a vector con el target más los atributos, de modo que la entrada es (48, `n_features`) y la LSTM ve la evolución conjunta de todas las variables.

Atributos. Usamos las **mismas cinco exógenas que el Lasso** (`wind_max`, `temperature`, `rh`, `mslp`, `wind_direction`), dejando la LSTM-exo comparable contra la **LSTM univariante** (misma arquitectura, sin exógenas) y el **Lasso** (mismos atributos, lineal). La pregunta repite la del Lasso, ¿aportan las exógenas o `wind_avg` ya contiene casi toda la señal predecible?

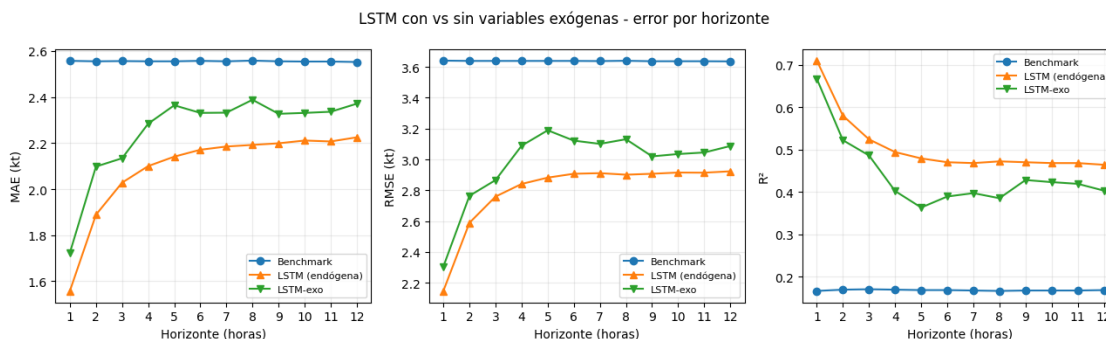
Decisiones de diseño (mínimas, para aislar el efecto de las exógenas):

- **Misma arquitectura ganadora** (LSTM(32) → Dropout(0.2) → Dense(12)), cambiando solo el ancho de entrada, así toda diferencia es atribuible a las exógenas.
- **Solo valores pasados** en la ventana; en el origen la red ve las últimas 48 h de las seis variables y proyecta 12 pasos, el mismo conjunto de información que el Lasso, sin fuga de datos.
- **Z-score por atributo** con estadísticos del train; desnormalizamos la salida con media y desvío de `wind_avg` para reportar en kt.
- *Salvedad:* `wind_direction` es circular y el Z-score no lo respeta; la dejamos cruda por consistencia con el Lasso (seno/coseno sería la mejora natural).

Construcción de las ventanas y la arquitectura. `ventanas_mv` arma, por origen, una ventana (48, 6) con las seis variables y un target (12,) con solo los 12 futuros de `wind_avg` (columna 0). `build_lstm_exo` es idéntica a la LSTM ganadora salvo `Input((48, 6))`. Mismo protocolo: `validation_split=0.1`, `EarlyStopping(patience=5)` con mejores pesos, semilla 99.

1.8.1 Evaluación de la LSTM-exo

La evaluamos con la **misma metodología** que la LSTM univariante: una pasada de `predict()`, desnormalización a kt y métricas por horizonte. Al compartir ventana ($W = 48$) y horizonte ($H = 12$), ambas se miden sobre los mismos orígenes, así que la comparación es directa.



1.8.2 Interpretación de los resultados

Comparamos la LSTM-exo contra la LSTM univariante y el benchmark. El resultado es nítido:

- **Las exógenas mejoran el ajuste pero empeoran la generalización.** En entrenamiento la LSTM-exo logró menor `val_loss` (0.567 vs 0.635), pero **en el test el orden se invierte**: peor en todos los horizontes (RMSE 2.98 vs 2.80 kt, R^2 0.44 vs 0.51), perdiendo ya en $h = 1$ (2.31 vs 2.15).
- **Es sobreajuste de manual.** Las cinco exógenas suman grados de libertad que la red usa para memorizar relaciones que no se sostienen en test. `wind_max` correlaciona fuerte con el target, pero `temperature`, `rh`, `mslp` y `wind_direction` aportan sobre todo ruido a resolución horaria.
- **Coincide con el Lasso.** El lineal multivariante tampoco saca ventaja de las exógenas (punto 8). La señal predecible de `wind_avg` vive casi por completo en su estructura autorregresiva y estacional; sumar atributos no la aumenta y en una red de alta capacidad la perjudica.

Lección metodológica: más entradas no implican mejor pronóstico. La menor `val_loss` podría engañar; es la evaluación honesta sobre el test la que revela que las exógenas no ayudan. Para el punto 8 conservamos la **LSTM univariante** como representante de las redes.

1.9 Comparación de modelos y elección

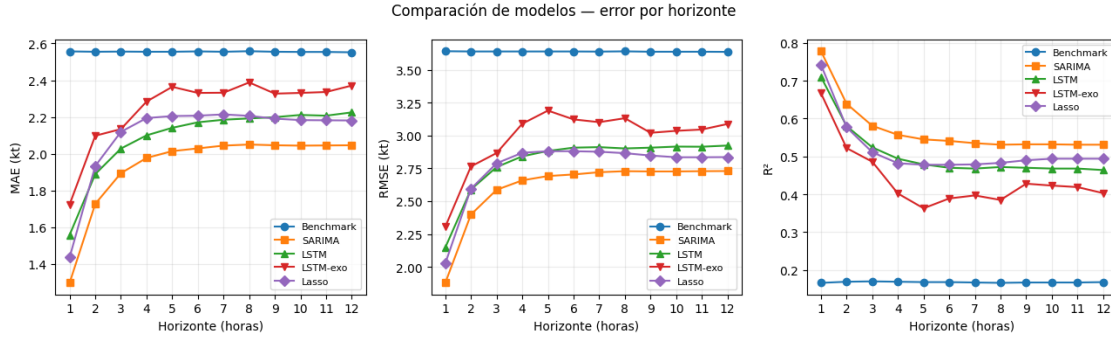
Reunimos los cinco modelos bajo la **misma metodología** (walk-forward por horizonte sobre el 20 % final, métricas en kt):

- **Benchmark**, persistencia estacional (punto 3),
- **SARIMA**(2,0,1)(1,0,1,24) (punto 4),
- **LSTM** univariante (punto 6),
- **LSTM-exo** con cinco exógenas (punto 6),
- **LassoCV** del primer entregable.

1.9.1 Re-evaluación del LassoCV bajo el mismo esquema

El LassoCV del entregable 1 se evaluó con validación cruzada sobre todo el dataset, no comparable con este walk-forward. Lo re-entrenamos (`StandardScaler` → `LassoCV` con `TimeSeriesSplit` para α) sobre el 80 % inicial y lo evaluamos sobre el 20 % final, por horizonte. Tres aclaraciones:

- **Es uno de los dos modelos con exógenas** (con la LSTM-exo). Usa 24 lags de `wind_avg`, las mismas cinco exógenas y la hora cíclica, todas conocidas en el origen. El benchmark, SARIMA y la LSTM simple son univariantes. La comparación responde si las exógenas justifican su costo frente a los univariantes.
- **Direct multi-step.** Como la LSTM, un modelo por horizonte (`objetivo = wind_avg.shift(-h)`), con un `gap` de 12 h entre train y test para no filtrar.
- **Conteos de orígenes algo distintos.** El ventaneo evalúa benchmark y SARIMA sobre 1218 orígenes, las dos LSTM sobre 1170 y el Lasso entre 1217 y 1228 según el horizonte; los sub-períodos se solapan casi por completo y los promedios siguen comparables.



1.9.2 Interpretación y elección del modelo

El gráfico y la tabla resumen (medias sobre los 12 horizontes) ordenan los cinco modelos:

- **Los cuatro modelos superan al benchmark en todos los horizontes.** La persistencia (RMSE 3.64 kt, R^2 0.17, plana) queda muy por debajo; las mejoras de RMSE promedio van del 18 % (LSTM-exo) al 28 % (SARIMA).
- **SARIMA es el mejor en todo el rango** (RMSE promedio 2.61 kt, R^2 0.57), con ventaja máxima en el corto plazo (en $h = 1$, R^2 0.78 vs 0.74 Lasso, 0.71 LSTM, 0.67 LSTM-exo).
- **Los modelos con exógenas no se despegan; la LSTM-exo es la peor.** Lasso (2.76 kt) y LSTM univariante (2.80) empatan por debajo de SARIMA, y la LSTM-exo queda última (2.98 kt, R^2 0.44). El hallazgo central, por partida doble, es que ni el lineal ni la red convierten las cinco exógenas en mejor pronóstico, y en la red incluso degradan el test (sobreajuste). La estructura autorregresiva y estacional de `wind_avg` ya contiene casi toda la señal.
- **A horizontes largos los modelos convergen** (~ 2.7 - 2.9 kt en $h = 12$): sin información de corto plazo, todos se apoyan en el ciclo diario.

¿Qué modelo para una fase operativa? **SARIMA**, por:

1. **Mejor desempeño en todos los horizontes**, con la mayor ventaja en las próximas horas, donde más se usa.
2. **Robustez operativa.** Univariante, solo necesita el histórico de `wind_avg`, sin depender de cinco sensores que pueden fallar; los modelos que sí los usan no sacan ventaja.
3. **Parsimonia e interpretabilidad.** Pocos parámetros, ajuste rápido e intervalos de predicción nativos, con la salvedad de la no-normalidad de los residuos, que los hace subestimar eventos extremos.
4. **Las redes no compensan su complejidad.** La univariante empata al Lasso y queda bajo SARIMA, y las exógenas la empeoran.

En síntesis, para esta serie y horizonte el modelo estocástico ofrece la mejor relación desempeño/simplicidad/robustez.